Project 1 Plan: PostgreSQL Docs Assistant with an Evaluation Harness

Nov 10, 2026 · @Jonas

Goal and scope

Build and deploy a documentation assistant that answers PostgreSQL questions with citations, and prove its quality with a measured evaluation harness. The plan is 6 weeks at 20 hours a week, 120 hours in total.

The finished project has four parts:

- **A RAG service**: ingestion, hybrid retrieval, reranking and cited answers behind a streaming API.
- **A small web UI**: chat with clickable source citations and a version selector.
- **An evaluation harness**: a labelled question set, retrieval and answer metrics, ablation experiments, and a CI gate that blocks regressions.
- **A deployed demo**: a live URL, a short architecture diagram, a README with measured results and a two-minute walkthrough video.

Out of scope: user accounts beyond a simple API key, fine-tuning, multi-agent workflows, and any claim that answers are production-grade.

Domain: PostgreSQL documentation

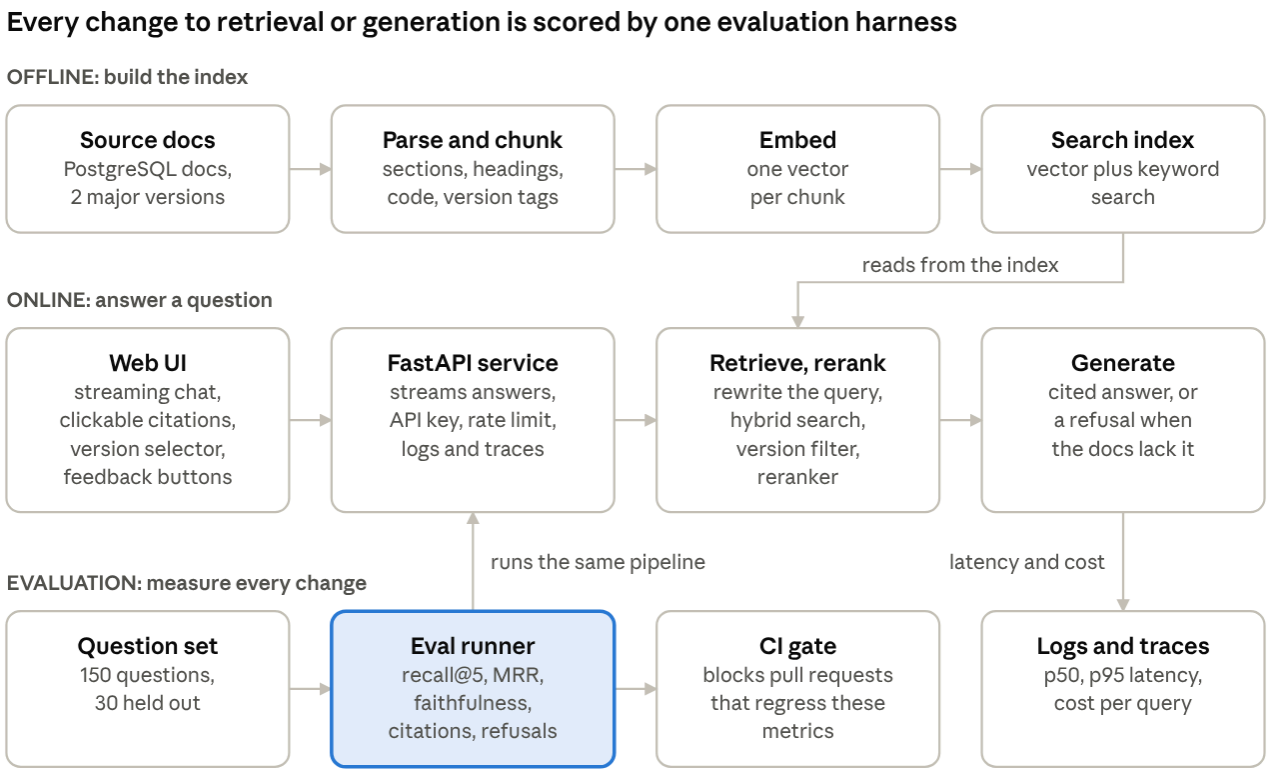
Use the public PostgreSQL documentation as the knowledge base. It suits this project for five reasons:

- **You can judge the answers.** As a software engineer you can verify correctness, which makes labelling the evaluation set fast and credible.
- **Real difficulty.** Content mixes prose, SQL code blocks, tables of settings and version-specific behaviour, so chunking and retrieval choices genuinely matter.
- **A built-in hard feature: versions.** The same page differs across major versions. A version selector forces version-aware retrieval and gives you evaluation cases that naive RAG fails.
- **Easy to get a labelled set.** Questions can be seeded from the docs themselves and from public Q&A threads, then verified by hand.
- **Recruiters can read it.** Any engineer can try the demo and tell immediately whether the answers are good.

Check the documentation's licence terms before you redistribute any text; index it, link to the source pages and keep citations pointing to the official site.

Alternative domain: a bilingual (Swahili and English) assistant over public-service or legal documents. It is more distinctive and locally relevant, but labelling needs domain expertise and wrong answers carry real risk. Treat it as a stretch follow-up, not the first build.

Architecture



system architecture · 3 bands, 12 components

Read it in three bands. The top band builds the search index once. The middle band answers each question through the API. The bottom band replays the question set through that same pipeline, so a pull request that lowers the metrics is blocked before it merges.

Evaluation plan

The evaluation harness is the part that makes this project stand out, so build it in week 2 and use it for every later decision. Freeze the question set before you tune anything, exactly as you froze the test set in HomeLens.

The question set (about 150 questions)

Slice	Count	Purpose
How-to and concept questions seeded from the docs, then verified by hand	70	Core quality
Version-specific questions (answer differs between major versions)	30	Tests version-aware retrieval
Exact-term questions (setting names, error codes, function names)	25	Tests keyword vs semantic retrieval
Unanswerable or out-of-scope questions	25	Tests refusal instead of hallucination

For each answerable question store: the question, the version, the gold source page or pages, and a short reference answer. Keep 30 questions as a held-out final set that you never tune on.

Metrics

Layer	Metric	What it tells you
Retrieval	Recall@5, MRR	Did the right page reach the model?
Generation	Faithfulness: share of answer claims supported by the cited text	Is the model making things up?
Generation	Answer correctness against the reference answer	Is it right?
Citations	Citation precision: share of cited sources that really support the claim	Are citations trustworthy?
Safety	Refusal rate on unanswerable questions	Does it say "I don't know"?
System	p50 and p95 latency, cost per query	Is it usable and affordable?

If you use an LLM as a judge for faithfulness and correctness, calibrate it first: hand-label 30 answers and report how often the judge agrees with you. State that agreement rate in the README.

Experiments (ablations)

Run one change at a time against the same question set and report a before/after table:

1. Chunk size and overlap (for example 300, 600 and 1,000 tokens).
2. Dense retrieval only vs keyword only vs hybrid.
3. With and without a reranker.
4. With and without query rewriting.
5. With and without version filtering.
6. Two different generator models, to show the cost-quality-latency trade-off.

CI gate

Run a fast subset of the evaluation on every pull request. Fail the build if recall@5, faithfulness or refusal rate drops below the thresholds you set after the baseline run. Publish the latest results table in the README automatically.

Week-by-week schedule

Six weeks at 20 hours a week. The evaluation harness comes in week 2, before any tuning, so every later change is measured.

Week	Focus	Hours split	Deliverable	Exit criterion
1	Corpus and naive baseline	6 scraping and parsing, 5 chunking v1, 4 embeddings and vector store, 5 CLI and repo setup	Docs for 2 major versions parsed into sections with headings, code blocks and version metadata; end-to-end CLI that answers with citations; repo with linting, tests and CI skeleton	You can ask a question in the terminal and get a cited answer
2	Evaluation harness and baseline	8 writing and verifying questions, 6 metric code, 4 judge calibration, 2 baseline report	150-question set (30 held out), retrieval and answer metrics, calibrated judge, baseline results table	Baseline numbers committed; question set frozen

Week	Focus	Hours split	Deliverable	Exit criterion
3	Retrieval upgrades	5 hybrid search, 4 reranker, 3 version filtering, 3 query rewriting, 5 ablation runs and write-up	Ablation table for chunk size, hybrid, reranker, rewriting and version filtering	Best configuration chosen from measured results, not guesses
4	Generation and API	5 prompt design with citations and refusal, 5 streaming FastAPI service, 3 API key and rate limiting, 4 logging and tracing, 3 tests and CI eval gate	Streaming API with structured logs and traces; CI blocks pull requests that regress the metrics	The API passes the CI evaluation gate
5	UI and deployment	7 web UI (streaming chat, clickable citations, version selector, feedback buttons), 6 Docker and cloud deployment, 4 monitoring of latency and cost, 3 load test	Live demo URL with health check and a basic dashboard	A stranger can open the URL and use it
6	Hardening and storytelling	5 failure analysis of the 20 worst answers, 4 model comparison table (cost, quality, latency), 6 README, diagram and 2-minute video, 3 write-up and resume bullets, 2 buffer	Public repo, results in the README, video walkthrough, short blog post	Everything in the stand-out checklist is ticked

Total: 120 hours. If you fall behind, use the cuts listed under risks, not the evaluation work.

Stand-out checklist

The project is finished when every item below is true.

- [ ] A live demo URL that works without signing in
- [ ] A README that opens with the problem, a screenshot and the headline results
- [ ] An architecture diagram and a short "decisions and trade-offs" section
- [ ] The ablation table, with one sentence per row on what you learned
- [ ] A frozen 150-question set in the repo, with the judge's measured agreement with your labels
- [ ] Cost per query and p50 and p95 latency, measured
- [ ] A CI badge, and a pull request that was blocked by the evaluation gate
- [ ] A "known failures" section with real examples from your worst 20 answers
- [ ] Tests for ingestion, retrieval and the API
- [ ] A two-minute video walkthrough
- [ ] Resume bullets with numbers, for example: "Built and deployed a RAG assistant over PostgreSQL docs; raised recall@5 from X to Y via hybrid search and reranking; cut p95 latency to Z s" (fill X, Y and Z from your own measured results)

Risks and what to cut

The most likely failure is spending weeks on the UI or on infrastructure and leaving the evaluation thin. Protect the evaluation work and cut elsewhere.

Risk	Likelihood	Mitigation
Parsing the docs takes longer than planned	High	Start with HTML section parsing only; ignore edge-case pages; index 2 versions, not 4
Labelling 150 questions is slow	Medium	Seed questions from the docs, verify quickly, and start with 100 if needed; never skip the unanswerable slice
LLM judge disagrees with you	Medium	Calibrate on 30 hand-labelled answers; report agreement; fall back to manual scoring on a smaller set
API and hosting costs grow	Medium	Cache answers, cap tokens, use a small model by default, set a hard monthly budget alert

Risk	Likelihood	Mitigation
Scope creep (agents, accounts, fine-tuning)	High	Keep to the out-of-scope list in the first section

Cut order if you fall behind: first the second generator-model comparison, then the monitoring dashboard, then the feedback buttons, then query rewriting. Never cut the question set, the baseline, the ablation table or the CI gate.

Open questions for you:

- Which cloud or hosting platform do you already know? That decides the deployment week.
- Do you prefer a hosted LLM API or a local open model? A hosted API is faster to build; a local model adds a stronger cost and latency story.
- Do you want a Swahili-and-English variant as a follow-up?